Red Hat OpenShift AI Self-Managed 3.0

Release notes

Features, enhancements, resolved issues, and known issues associated with this release

Last Updated: 2025-12-03

Red Hat OpenShift AI Self-Managed 3.0 Release notes

Features, enhancements, resolved issues, and known issues associated with this release

Legal Notice

Abstract

These release notes provide an overview of new features, enhancements, resolved issues, and known issues in version 3.0 of Red Hat OpenShift AI.

Table of Contents

CHAPTER 1. UPGRADE TO OPENSHIFT AI 3.0 NOT SUPPORTED

You cannot upgrade from OpenShift AI 2.25 or any earlier version to 3.0. OpenShift AI 3.0 introduces significant technology and component changes and is intended for new installations only. To use OpenShift AI 3.0, install the Red Hat OpenShift AI Operator on a cluster running OpenShift Container Platform 4.19 or later and select the **fast-3.x** channel.

Support for upgrades will be available in a later release, including upgrades from OpenShift AI 2.25 to a stable 3.x version.

For more information, see the Why upgrades to OpenShift AI 3.0 are not supported Knowledgebase article.

CHAPTER 2. OVERVIEW OF OPENSHIFT AI

Red Hat OpenShift AI is a platform for data scientists and developers of artificial intelligence and machine learning (AI/ML) applications.

OpenShift AI provides an environment to develop, train, serve, test, and monitor AI/ML models and applications on-premise or in the cloud.

For data scientists, OpenShift AI includes Jupyter and a collection of default workbench images optimized with the tools and libraries required for model development, and the TensorFlow and PyTorch frameworks. Deploy and host your models, integrate models into external applications, and export models to host them in any hybrid cloud environment. You can enhance your projects on OpenShift AI by building portable machine learning (ML) workflows with AI pipelines by using Docker containers. You can also accelerate your data science experiments through the use of graphics processing units (GPUs) and Intel Gaudi AI accelerators.

For administrators, OpenShift AI enables data science workloads in an existing Red Hat OpenShift or ROSA environment. Manage users with your existing OpenShift identity provider, and manage the resources available to workbenches to ensure data scientists have what they require to create, train, and host models. Use accelerators to reduce costs and allow your data scientists to enhance the performance of their end-to-end data science workflows using graphics processing units (GPUs) and Intel Gaudi AI accelerators.

OpenShift AI has two deployment options:

- **Self-managed software** that you can install on-premise or in the cloud. You can install OpenShift AI Self-Managed in a self-managed environment such as OpenShift Container Platform, or in Red Hat-managed cloud environments such as Red Hat OpenShift Dedicated (with a Customer Cloud Subscription for AWS or GCP), Red Hat OpenShift Service on Amazon Web Services (ROSA classic or ROSA HCP), or Microsoft Azure Red Hat OpenShift.

- A **managed cloud service**, installed as an add-on in Red Hat OpenShift Dedicated (with a Customer Cloud Subscription for AWS or GCP) or in Red Hat OpenShift Service on Amazon Web Services (ROSA classic).
  For information about OpenShift AI Cloud Service, see Product Documentation for Red Hat OpenShift AI.

For information about OpenShift AI supported software platforms, components, and dependencies, see the Supported Configurations for 3.x Knowledgebase article.

For a detailed view of the 3.0 release lifecycle, including the full support phase window, see the Red Hat OpenShift AI Self-Managed Life Cycle Knowledgebase article.

CHAPTER 3. NEW FEATURES AND ENHANCEMENTS

This section describes new features and enhancements in Red Hat OpenShift AI 3.0.

3.1. NEW FEATURES

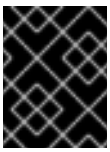
PyTorch v2.8.0 KFTO training images now generally available

You can now use PyTorch v2.8.0 training images for distributed workloads in OpenShift AI.
The following new images are available:

- **ROCm-compatible KFTO training image: quay.io/modh/training:py312-rocm63-torch280** Compatible with AMD accelerators supported by ROCm 6.3.
- **CUDA-compatible KFTO training image: quay.io/modh/training:py312-cuda128-torch280** Compatible with NVIDIA GPUs supported by CUDA 12.8.

Hardware profiles

A new Hardware Profiles feature replaces the previous Accelerator Profiles and legacy **Container Size** selector for workbenches.
Hardware profiles provide a more flexible and consistent way to define compute configurations for AI workloads, simplifying resource management across different hardware types.



IMPORTANT

The Accelerator Profiles and legacy Container Size selector are now deprecated and will be removed in a future release.

Connections API now generally available

The **Connections API** is now available as a general availability (GA) feature in OpenShift AI.
This API enables you to create and manage connections to external data sources and services directly within OpenShift AI. Connections are stored as Kubernetes Secrets with standardized annotations, allowing protocol-based validation and routing across integrated components.

IBM Power and IBM Z architecture support

OpenShift AI now supports the IBM Power (ppc64le) and IBM Z (s390x) architectures.
This expanded platform support enables deployment of AI and machine learning workloads on IBM enterprise hardware, providing greater flexibility and scalability across heterogeneous environments.

For more information about supported software platforms, components, and dependencies, see the Knowledgebase article: Supported Configurations for 3.x.

IBM Power and IBM Z architecture support for TrustyAI

TrustyAI is now available as a general availability (GA) feature for the IBM Power (ppc64le) and IBM Z (s390x) architectures.
TrustyAI is an open-source Responsible AI toolkit that provides a suite of tools to support responsible and transparent AI workflows. It offers capabilities such as fairness and data drift metrics, local and global model explanations, text detoxification, language model benchmarking, and language model guardrails.

These capabilities help ensure transparency, accountability, and the ethical use of AI systems within OpenShift AI environments on IBM Power and IBM Z systems.

IBM Power and IBM Z architecture support for Model Registry

Model Registry is now available for the IBM Power (ppc64le) and IBM Z (s390x) architectures.
Model Registry is an open-source component that simplifies and standardizes the management of AI and machine learning (AI/ML) model lifecycles. It provides a centralized platform for storing, versioning, and governing models, enabling seamless collaboration across data science and MLOps teams.

Model Registry supports capabilities such as model versioning and lineage tracking, metadata management and model discovery, model approval and promotion workflows, integration with CI/CD and deployment pipelines, and governance, auditability, and compliance features on IBM Power and IBM Z systems.

IBM Power and IBM Z architecture support for Notebooks

Notebooks are now available for the IBM Power (ppc64le) and IBM Z (s390x) architectures.
Notebooks provide containerized, browser-based development environments for data science, machine learning, research, and coding within the OpenShift AI ecosystem. These environments can be launched through Workbenches and include the following options:

- **Jupyter Minimal notebook**: A lightweight JupyterLab IDE for basic Python development and model prototyping.
- **Jupyter Data Science notebook**: Preconfigured with popular data science libraries and tools for end-to-end workflows.
- **Jupyter TrustyAI notebook**: An environment for Responsible AI tasks, including model explainability, fairness, data drift detection, and text detoxification.
- **Code Server**: A browser-based VS Code environment for collaborative development with familiar IDE features.
- **Runtime Minimal** and **Runtime Data Science**: Headless environments for automated workflows and consistent pipeline execution.

IBM Power architecture support for Feature Store

Feature Store is now supported on the IBM Power (ppc64le) architecture.
This support enables users to build, register, and manage features for machine learning models directly on IBM Power-based environments, with full integration with OpenShift AI.

IBM Power architecture support for AI Pipelines

AI Pipelines are now supported on the IBM Power (ppc64le) architecture.
This capability enables users to define, run, and monitor AI pipelines natively within OpenShift AI, leveraging the performance and scalability of IBM Power systems for AI workloads.

AI Pipelines executed on IBM Power systems maintain functional parity with x86 deployments.

Support for IBM Power accelerated Triton Inference Server

You can now enable Power architecture support for Triton inference server (CPU only) with FIL, PyTorch, Python and ONNX backend. You can deploy Triton inference server as a custom model serving runtime on IBM Power architecture in Red Hat OpenShift AI.
For details, see Triton Inference Server image .

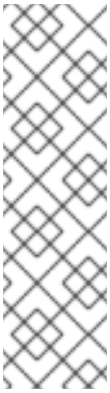
Support for IBM Z accelerated Triton Inference Server

You can now enable Z architecture support for the Triton Inference Server (Telum I/Telum II) with multiple backend options, including ONNX-MLIR, Snap ML (C++), and PyTorch. The Triton Inference Server can be deployed as a custom model serving runtime on IBM Z architecture as a Technology Preview feature in Red Hat OpenShift AI.
For details, see IBM Z accelerated Triton Inference Server .

IBM Spyre AI Accelerator model serving support on IBM Z platforms

Model serving with the IBM Spyre AI Accelerator is now available as a general availability (GA) feature for IBM Z platforms.
The IBM Spyre Operator automates installation and integrates key components such as the device plugin, secondary scheduler, and monitoring.

For more information, see the IBM Spyre Operator catalog entry: IBM Spyre Operator – Red Hat Ecosystem Catalog.



NOTE

On IBM Z and IBM LinuxONE, Red Hat OpenShift AI supports deploying large language models (LLMs) with vLLM on IBM Spyre. The Triton Inference Server is supported on Telum (CPU) only.

For more information, see the following documentation:

- Triton Inference Server
- vLLM on Spyre

Model customization components

OpenShift AI 3.0 introduces a suite of **model customization components** that streamline and enhance the process of preparing, fine-tuning, and deploying AI models.
The following components are now available:

- **Red Hat AI Python Index** A Red Hat-maintained Python package index that hosts supported builds of packages useful for AI and machine learning notebooks. Using the Red Hat AI Python Index ensures reliable and secure access to these packages in both connected and disconnected environments.
- **docling**: A powerful Python library for advanced data processing that converts unstructured documents, such as PDFs or images, into clean, machine-readable formats for AI and ML workloads.
- **Synthetic Data Generation Hub (sdg-hub)** A toolkit for generating high-quality synthetic data to augment datasets, improve model robustness, and address edge cases.
- **Training Hub**: A framework that simplifies and accelerates the fine-tuning and customization of foundation models by using your own data.

- **Kubeflow Trainer**: A Kubernetes-native capability that enables distributed training and fine-tuning of models while abstracting the underlying infrastructure complexity.
- **AI Pipelines**: A Kubeflow-native capability for building configurable workflows across AI components, including all other model customization modules in this suite.

3.2. ENHANCEMENTS

Hybrid search support for remote vector databases in Llama Stack

You can now enable hybrid search on remote vector databases in Llama Stack in OpenShift AI. This enhancement allows enterprises to use their existing managed vector database infrastructure while maintaining high retrieval performance and flexibility across different database types.

IBM Spyre support for IBM Z with Caikit-TGIS adapter

You can now serve models with IBM Spyre AI accelerators on IBM Z (s390x architecture) by using the **vLLM Spyre s390x ServingRuntime for KServe**with the **Caikit-TGIS gRPC adapter**.
This integration enables high-performance model serving and inference for generative AI workloads on IBM Z systems within OpenShift AI.

Data Science Pipelines renamed to AI Pipelines

OpenShift AI now uses the term "AI Pipelines" instead of "Data Science Pipelines" to better reflect the broader range of AI and generative AI use cases supported by the platform.
In the default **DataScienceCluster** (**default-dsc**), the **datasciencepipelines** component has been renamed to **aipipelines** to align with this terminology update.

This is a naming change only. The AI pipelines functionality remains the same.

Model Catalog enhancements with model validation data

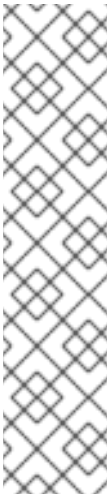
The **Model Details** page in the OpenShift AI Model Catalog now includes comprehensive model validation data, such as performance benchmarks, hardware compatibility, and other key metrics.
This enhancement provides a unified and detailed view consistent with the Jounce UI model details layout, enabling users to evaluate models more effectively from a single interface.

Model Catalog performance data with search and filtering

The Model Catalog now includes detailed performance and validation data for Red Hat-validated third-party models, such as benchmarks and hardware compatibility metrics.
Enhanced search and filtering capabilities, such as filtering by latency or hardware profile, help users quickly identify models optimized for their specific use cases and available resources, providing a unified discovery experience within the Red Hat AI Hub.

Distributed Inference with llm-d is now generally available (GA)

Distributed Inference with llm-d supports multi-model serving, intelligent inference scheduling, and disaggregated serving for improved GPU utilization on generative AI models.



NOTE

The following capabilities are not fully supported:

- Wide Expert-Parallelism multi-node: Developer Preview.
- Wide Expert-Parallelism on Blackwell B200: Not available but can be provided as a Technology Preview.
- Multi-node on GB200: Not supported.
- Gateway discovery and association are not supported in the UI during model deployment in this release. Users must associate models with Gateways by applying the resource manifests directly through the API or CLI.

User interface for Distributed Inference with llm-d deployment configuration

OpenShift AI now includes a user interface (UI) for configuring large language model (LLM) deployments that run on the **llm-d** Serving Runtime.

This streamlined interface simplifies common deployment scenarios by providing essential configuration options with sensible defaults while still allowing explicit selection of the **llm-d** runtime for your deployment.

The new UI reduces setup complexity and helps users deploy distributed inference workloads more efficiently.

New navigation system

OpenShift AI 3.0 introduces a redesigned, streamlined navigation system that improves usability and workflow efficiency.

The new layout enables users to move seamlessly between features, simplifying access to key capabilities and supporting a smoother end-to-end experience.

Enhanced authentication for AI Pipelines

OpenShift AI 3.0 replaces **oauth-proxy** with **kube-rbac-proxy** for AI Pipelines as part of the platform-wide authentication transition.

This update improves security and compatibility, particularly for environments without an internal OAuth server, such as Red Hat OpenShift Service on AWS.

When migrating to **kube-rbac-proxy**, SubjectAccessReview (SAR) requirements and RBAC permissions change accordingly. Users who rely on the built-in **ds-pipeline-user-access-<dspa-name>** role are updated automatically, while others must ensure their roles include access to the **datasciencepipelinesapplications/api** subresource with the following verbs: **create**, **update**, **patch**, **delete**, **get**, **list**, and **watch**.

Observability and Grafana integration for Distributed Inference with llm-d

In OpenShift AI 3.0, platform administrators can connect observability components to Distributed Inference with llm-d deployments and integrate with self-hosted Grafana instances to monitor inference workloads.

This capability allows teams to collect and visualize Prometheus metrics from Distributed Inference with llm-d for performance analysis and custom dashboard creation.

CHAPTER 4. TECHNOLOGY PREVIEW FEATURES

IMPORTANT

This section describes Technology Preview features in Red Hat OpenShift AI 3.0. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see Technology Preview Features Support Scope .

TrustyAI–Llama Stack integration for safety, guardrails, and evaluation

You can now use the Guardrails Orchestrator from TrustyAI with Llama Stack as a Technology Preview feature.
This integration enables built-in detection and evaluation workflows to support AI safety and content moderation. When TrustyAI is enabled and the FMS Orchestrator and detectors are configured, no manual setup is required.

To activate this feature, set the following field in the **DataScienceCluster** custom resource for the OpenShift AI Operator: **spec.llamastackoperator.managementState: Managed**

For more information, see the TrustyAI FMS Provider on GitHub: TrustyAI FMS Provider.

AI Available Assets page for deployed models and MCP servers

A new **AI Available Assets** page enables AI engineers and application developers to view and consume deployed AI resources within their projects.
This enhancement introduces a filterable UI that lists available models and Model Context Protocol (MCP) servers in the selected project, allowing users with appropriate permissions to identify accessible endpoints and integrate them directly into the AI Playground or other applications.

Generative AI Playground for model testing and evaluation

The Generative AI (GenAI) Playground introduces a unified, interactive experience within the OpenShift AI dashboard for experimenting with foundation and custom models.
Users can test prompts, compare models, and evaluate Retrieval-Augmented Generation (RAG) workflows by uploading documents and chatting with their content. The GenAI Playground also supports integration with approved Model Context Protocol (MCP) servers and enables export of prompts and agent configurations as runnable code for continued iteration in local IDEs.

Chat context is preserved within each session, providing a suitable environment for prompt engineering and model experimentation.

Support for air-gapped Llama Stack deployments

You can now install and operate Llama Stack and RAG/Agentic components in fully disconnected (air-gapped) OpenShift AI environments.
This enhancement enables secure deployment of Llama Stack features without internet access, allowing organizations to use AI capabilities while maintaining compliance with strict network security policies.

Feature Store integration with Workbenches and new user access capabilities

This feature is available as a Technology Preview.
The Feature Store is now integrated with OpenShift AI, data science projects, and workbenches. This integration also introduces centrally managed, role-based access control (RBAC) capabilities for improved governance.

These enhancements provide two key capabilities:

- Feature development within the workbench environment.
- Administrator-controlled user access.
  This update simplifies and accelerates feature discovery and consumption for data scientists while allowing platform teams to maintain full control over infrastructure and feature access.

Feature Store user interface

The **Feature Store** component now includes a web-based user interface (UI).
You can use the UI to view registered Feature Store objects and their relationships, such as features, data sources, entities, and feature services.

To enable the UI, edit your **FeatureStore** custom resource (CR) instance. When you save the change, the Feature Store Operator starts the UI container and creates an OpenShift route for access.

For more information, see *Setting up the Feature Store user interface for initial use* .

IBM Spyre AI Accelerator model serving support on x86 platforms

Model serving with the IBM Spyre AI Accelerator is now available as a Technology Preview feature for x86 platforms. The IBM Spyre Operator automates installation and integrates the device plugin, secondary scheduler, and monitoring. For more information, see the IBM Spyre Operator catalog entry.

Build Generative AI Apps with Llama Stack on OpenShift AI

With this release, the Llama Stack Technology Preview feature enables Retrieval-Augmented Generation (RAG) and agentic workflows for building next-generation generative AI applications. It supports remote inference, built-in embeddings, and vector database operations. It also integrates with providers like TrustyAI's provider for safety and Trusty AI's LM-Eval provider for evaluation. This preview includes tools, components, and guidance for enabling the Llama Stack Operator, interacting with the RAG Tool, and automating PDF ingestion and keyword search capabilities to enhance document discovery.

Centralized platform observability

Centralized platform observability, including metrics, traces, and built-in alerts, is available as a Technology Preview feature. This solution introduces a dedicated, pre-configured observability stack for OpenShift AI that allows cluster administrators to perform the following actions:

- View platform metrics (Prometheus) and distributed traces (Tempo) for OpenShift AI components and workloads.
- Manage a set of built-in alerts (alertmanager) that cover critical component health and performance issues.

- Export platform and workload metrics to external 3rd party observability tools by editing the **DataScienceClusterInitialization** (DSCI) custom resource.
  You can enable this feature by integrating with the Cluster Observability Operator, Red Hat build of OpenTelemetry, and Tempo Operator. For more information, see Monitoring and observability. For more information, see Managing observability.

Support for Llama Stack Distribution version 0.3.0

The Llama Stack Distribution now includes version 0.3.0 as a Technology Preview feature.
This update introduces several enhancements, including expanded support for retrieval-augmented generation (RAG) pipelines, improved evaluation provider integration, and updated APIs for agent and vector store management. It also provides compatibility updates aligned with recent OpenAI API extensions and infrastructure optimizations for distributed inference.

The previously supported version was 0.2.22.

Support for Kubernetes Event-driven Autoscaling (KEDA)

OpenShift AI now supports Kubernetes Event-driven Autoscaling (KEDA) in its KServe RawDeployment mode. This Technology Preview feature enables metrics-based autoscaling for inference services, allowing for more efficient management of accelerator resources, reduced operational costs, and improved performance for your inference services.
To set up autoscaling for your inference service in KServe RawDeployment mode, you need to install and configure the OpenShift Custom Metrics Autoscaler (CMA), which is based on KEDA.

For more information about this feature, see: Configuring metrics-based autoscaling.

LM-Eval model evaluation UI feature

TrustyAI now offers a user-friendly UI for LM-Eval model evaluations as Technology Preview. This feature allows you to input evaluation parameters for a given model and returns an evaluation-results page, all from the UI.

Use Guardrails Orchestrator with LlamaStack

You can now run detections using the Guardrails Orchestrator tool from TrustyAI with Llama Stack as a Technology Preview feature, using the built-in detection component. To use this feature, ensure TrustyAI is enabled, the FMS Orchestrator and detectors are set up, and KServe RawDeployment mode is in use for full compatibility if needed. There is no manual set up required. Then, in the **DataScienceCluster** custom resource for the Red Hat OpenShift AI Operator, set the **spec.llamastackoperator.managementState** field to **Managed**.
For more information, see Trusty AI FMS Provider on GitHub.

Support for creating and managing Ray Jobs with the CodeFlare SDK

You can now create and manage Ray Jobs on Ray Clusters directly through the CodeFlare SDK. This enhancement aligns the CodeFlare SDK workflow with the KubernetesFlow Training Operator (KFTO) model, where a job is created, run, and completed automatically. This enhancement simplifies manual cluster management by preventing Ray Clusters from remaining active after job completion.

Support for direct authentication with an OIDC identity provider

Direct authentication with an OpenID Connect (OIDC) identity provider is now available as a Technology Preview feature.

This enhancement centralizes OpenShift AI service authentication through the Gateway API, providing a secure, scalable, and manageable authentication model. You can configure the Gateway API with your external OIDC provider by using the **GatewayConfig** custom resource.

Custom flow estimator for Synthetic Data Generation pipelines

You can now use a custom flow estimator for synthetic data generation (SDG) pipelines.
For supported and compatible tagged SDG teacher models, the estimator helps you evaluate a chosen teacher model, custom flow, and supported hardware on a sample dataset before running full workloads.

Llama Stack support and optimization for single node OpenShift (SNO)

Llama Stack core can now deploy and run efficiently on single node OpenShift (SNO).
This enhancement optimizes component startup and resource usage so that Llama Stack can operate reliably in single-node cluster environments.

FAISS vector storage integration

You can now use the FAISS (Facebook AI Similarity Search) library as an inline vector store in OpenShift AI.
FAISS is an open-source framework for high-performance vector search and clustering, optimized for dense numerical embeddings with both CPU and GPU support. When enabled with an embedded SQLite backend in the Llama Stack Distribution, FAISS stores embeddings locally within the container, removing the need for an external vector database service.

New Feature Store component

You can now install and manage Feature Store as a configurable component in OpenShift AI. Based on the open-source Feast project, Feature Store acts as a bridge between ML models and data, enabling consistent and scalable feature management across the ML lifecycle.
This Technology Preview release introduces the following capabilities:

- Centralized feature repository for consistent feature reuse
- Python SDK and CLI for programmatic and command-line interactions to define, manage, and retrieve features for ML models
- Feature definition and management
- Support for a wide range of data sources
- Data ingestion via feature materialization
- Feature retrieval for both online model inference and offline model training
- Role-Based Access Control (RBAC) to protect sensitive features
- Extensibility and integration with third-party data and compute providers
- Scalability to meet enterprise ML needs
- Searchable feature catalog
- Data lineage tracking for enhanced observability
  For configuration details, see Configuring Feature Store.

FIPS support for Llama Stack and RAG deployments

You can now deploy Llama Stack and RAG or agentic solutions in regulated environments that require FIPS compliance.
This enhancement provides FIPS-certified and compatible deployment patterns to help organizations meet strict regulatory and certification requirements for AI workloads.

Validated sdg-hub notebooks for Red Hat AI Platform

Validated **sdg_hub** example notebooks are now available to provide a notebook-driven user experience in OpenShift AI 3.0.
These notebooks support multiple Red Hat platforms and enable customization through SDG pipelines. They include examples for the following use cases:

- Knowledge and skills tuning, including annotated examples for fine-tuning models.
- Synthetic data generation with reasoning traces to customize reasoning models.
- Custom SDG pipelines that demonstrate using default blocks and creating new blocks for specialized workflows.

RAGAS evaluation provider for Llama Stack (inline and remote)

You can now use the Retrieval-Augmented Generation Assessment (RAGAS) evaluation provider to measure the quality and reliability of RAG systems in OpenShift AI.
RAGAS provides metrics for retrieval quality, answer relevance, and factual consistency, helping you identify issues and optimize RAG pipeline configurations.

The integration with the Llama Stack evaluation API supports two deployment modes:

- Inline provider: Runs RAGAS evaluation directly within the Llama Stack server process.
- Remote provider: Runs RAGAS evaluation as distributed jobs using OpenShift AI pipelines. The RAGAS evaluation provider is now included in the Llama Stack distribution.

Enable targeted deployment of workbenches to specific worker nodes in Red Hat OpenShift AI Dashboard using node selectors

Hardware profiles are now available as a Technology Preview. The hardware profiles feature enables users to target specific worker nodes for workbenches or model-serving workloads. It allows users to target specific accelerator types or CPU-only nodes.
This feature replaces the current accelerator profiles feature and container size selector field, offering a broader set of capabilities for targeting different hardware configurations. While accelerator profiles, taints, and tolerations provide some capabilities for matching workloads to hardware, they do not ensure that workloads land on specific nodes, especially if some nodes lack the appropriate taints.

The hardware profiles feature supports both accelerator and CPU-only configurations, along with node selectors, to enhance targeting capabilities for specific worker nodes. Administrators can configure hardware profiles in the settings menu. Users can select the enabled profiles using the UI for workbenches, model serving, and AI pipelines where applicable.

RStudio Server workbench image

With the RStudio Server workbench image, you can access the RStudio IDE, an integrated development environment for R. The R programming language is used for statistical computing and graphics to support data analysis and predictions.

To use the RStudio Server workbench image, you must first build it by creating a secret and triggering the **BuildConfig**, and then enable it in the OpenShift AI UI by editing the **rstudio-rhel9** image stream. For more information, see Building the RStudio Server workbench images .

IMPORTANT

Disclaimer: Red Hat supports managing workbenches in OpenShift AI. However, Red Hat does not provide support for the RStudio software. RStudio Server is available through rstudio.org and is subject to their licensing terms. You should review their licensing terms before you use this sample workbench.

CUDA - RStudio Server workbench image

With the CUDA - RStudio Server workbench image, you can access the RStudio IDE and NVIDIA CUDA Toolkit. The RStudio IDE is an integrated development environment for the R programming language for statistical computing and graphics. With the NVIDIA CUDA toolkit, you can enhance your work by using GPU-accelerated libraries and optimization tools.
To use the CUDA - RStudio Server workbench image, you must first build it by creating a secret and triggering the **BuildConfig**, and then enable it in the OpenShift AI UI by editing the **rstudio-rhel9** image stream. For more information, see Building the RStudio Server workbench images .

IMPORTANT

Disclaimer: Red Hat supports managing workbenches in OpenShift AI. However, Red Hat does not provide support for the RStudio software. RStudio Server is available through rstudio.org and is subject to their licensing terms. You should review their licensing terms before you use this sample workbench.

The CUDA - RStudio Server workbench image contains NVIDIA CUDA technology. CUDA licensing information is available in the CUDA Toolkit documentation. You should review their licensing terms before you use this sample workbench.

Support for multinode deployment of very large models

Serving models over multiple graphical processing unit (GPU) nodes when using a single-model serving runtime is now available as a Technology Preview feature. Deploy your models across multiple GPU nodes to improve efficiency when deploying large models such as large language models (LLMs). For more information, see Deploying models by using multiple GPU nodes .

CHAPTER 5. DEVELOPER PREVIEW FEATURES

IMPORTANT

This section describes Developer Preview features in Red Hat OpenShift AI 3.0. Developer Preview features are not supported by Red Hat in any way and are not functionally complete or production-ready. Do not use Developer Preview features for production or business-critical workloads. Developer Preview features provide early access to functionality in advance of possible inclusion in a Red Hat product offering. Customers can use these features to test functionality and provide feedback during the development process. Developer Preview features might not have any documentation, are subject to change or removal at any time, and have received limited testing. Red Hat might provide ways to submit feedback on Developer Preview features without an associated SLA.

For more information about the support scope of Red Hat Developer Preview features, see Developer Preview Support Scope.

Model-as-a-Service (MaaS) integration

This feature is available as a Developer Preview.
OpenShift AI now includes Model-as-a-Service (MaaS) to address resource consumption and governance challenges associated with serving large language models (LLMs).

MaaS provides centralized control over model access and resource usage by exposing models through managed API endpoints, allowing administrators to enforce consumption policies across teams.

This Developer Preview introduces the following capabilities:

- Policy and quota management
- Authentication and authorization
- Usage tracking
- User management
  For more information, see Introducing Models-as-a-Service in OpenShift AI .

AI Available Assets integration with Model-as-a-Service (MaaS)

This feature is available as a Developer Preview.
You can now access and consume Model-as-a-Service (MaaS) models directly from the **AI Available Assets** page in the GenAI Studio.

Administrators can configure a MaaS by enabling the toggle in the **Model Deployments** page. When a model is marked as a service, it becomes global and visible across all projects in the cluster.

Additional fields added to Model Deployments for AI Available Assets integration

This feature is available as a Developer Preview.
Administrators can now add metadata to models during deployment so that they are automatically listed on the **AI Available Assets** page.

The following table describes the new metadata fields that streamline the process of making models discoverable and consumable by other teams:

Field name	Field type	Description
Use Case	Free-form text	Describes the model's primary purpose, for example, "Customer Churn Prediction" or "Image Classification for Product Catalog."
Description	Free-form text	Provides more detailed context and functionality notes for the model.
Add to AI Assets	Checkbox	When enabled, automatically publishes the model and its metadata to the **AI Available Assets** page.

Compatibility of Llama Stack remote providers and SDK with MCP HTTP streaming protocol

This feature is available as a Developer Preview.
Llama Stack remote providers and the SDK are now compatible with the Model Context Protocol (MCP) HTTP streaming protocol.

This enhancement enables developers to build fully stateless MCP servers, simplify deployment on standard Llama Stack infrastructure (including serverless environments), and improve scalability. It also prepares for future enhancements such as connection resumption and provides a smooth transition away from Server-Sent Events (SSE).

Packaging of ITS Hub dependencies to the Red Hat–maintained Python index

This feature is available as a Developer Preview.
All Inference Time Scaling (ITS) runtime dependencies are now packaged in the Red Hat-maintained Python index, allowing Red Hat AI and OpenShift AI customers to install **its_hub** and its dependencies directly by using **pip**.

This enhancement enables users to build custom inference images with ITS algorithms focused on improving model accuracy at inference time without requiring model retraining, such as:

- Particle filtering
- Best-of-N
- Beam search
- Self-consistency
- Verifier or PRM-guided search
  For more information, see the ITS Hub on GitHub.

Dynamic hardware-aware continual training strategy

Static hardware profile support is now available to help users select training methods, models, and hyperparameters based on VRAM requirements and reference benchmarks. This approach ensures predictable and reliable training workflows without dynamic hardware discovery.
The following components are included:

- **API Memory Estimator**: Accepts model, training method, dataset metadata, and assumed hyperparameters as input and returns an estimated VRAM requirement for the training job. Delivered as an API within Training Hub.
- **Reference Profiles and Benchmarks**: Provides end-to-end training time benchmarks for OpenShift AI Innovation (OSFT) and Performance Team (LAB SFT) baselines, delivered as static tables and documentation in Training Hub.
- **Hyperparameter Guidance**: Publishes safe starting ranges for key hyperparameters such as learning rate, batch size, epochs, and LoRA rank. Integrated into example notebooks maintained by the AI Innovation team.

IMPORTANT

Hardware discovery is not included in this release. Only static reference tables and guidance are provided; automated GPU or CPU detection is not yet supported.

Human-in-the-Loop (HIL) functionality in the Llama Stack agent

Human-in-the-Loop (HIL) functionality has been added to the Llama Stack agent to allow users to approve unread tool calls before execution.
This enhancement includes the following capabilities:

- Users can approve or reject unread tool calls through the responses API.
- Configuration options specify which tool calls require HIL approval.
- Tool calls pause until user approval is received for HIL-enabled tools.
- Tool calls that do not require HIL continue to run without interruption.

CHAPTER 6. SUPPORT REMOVALS

This section describes major changes in support for user-facing features in Red Hat OpenShift AI. For information about OpenShift AI supported software platforms, components, and dependencies, see the Supported Configurations for 3.x Knowledgebase article.

6.1. DEPRECATED

Deprecated annotation format for Connection Secrets

Starting with OpenShift AI 3.0, the **opendatahub.io/connection-type-ref** annotation format for creating Connection Secrets is deprecated.
For all new Connection Secrets, use the **opendatahub.io/connection-type-protocol** annotation instead. While both formats are currently supported, **connection-type-protocol** takes precedence and should be used for future compatibility.

6.1.1. Deprecated Kubeflow Training operator v1

The Kubeflow Training Operator (v1) is deprecated starting OpenShift AI 2.25 and is planned to be removed in a future release. This deprecation is part of our transition to Kubeflow Trainer v2, which delivers enhanced capabilities and improved functionality.

6.1.2. Deprecated TrustyAI service CRD v1alpha1

Starting with OpenShift AI 2.25, the **v1apha1** version is deprecated and planned for removal in an upcoming release. You must update the TrustyAI Operator to version **v1** to receive future Operator updates.

6.1.3. Deprecated KServe Serverless deployment mode

Starting with OpenShift AI 2.25, The KServe Serverless deployment mode is deprecated. You can continue to deploy models by migrating to the KServe RawDeployment mode. If you are upgrading to Red Hat OpenShift AI 3.0, all workloads that use the retired Serverless or ModelMesh modes must be migrated before upgrading.

6.1.4. Deprecated model registry API v1alpha1

Starting with OpenShift AI 2.24, the model registry API version **v1alpha1** is deprecated and will be removed in a future release of OpenShift AI. The latest model registry API version is **v1beta1**.

6.1.5. Multi-model serving platform (ModelMesh)

Starting with OpenShift AI version 2.19, the multi-model serving platform based on ModelMesh is deprecated. You can continue to deploy models on the multi-model serving platform, but it is recommended that you migrate to the single-model serving platform.

For more information or for help on using the single-model serving platform, contact your account manager.

6.1.6. Accelerator Profiles and legacy Container Size selector deprecated

Starting with OpenShift AI 3.0, Accelerator Profiles and the **Container Size** selector for workbenches are deprecated.

+ These features are replaced by the more flexible and unified Hardware Profiles capability.

6.1.7. Deprecated OpenVINO Model Server (OVMS) plugin

The CUDA plugin for the OpenVINO Model Server (OVMS) is now deprecated and will no longer be available in future releases of OpenShift AI.

6.1.8. OpenShift AI dashboard user management moved from **OdhDashboardConfig** to **Auth** resource

Previously, cluster administrators used the **groupsConfig** option in the **OdhDashboardConfig** resource to manage the OpenShift groups (both administrators and non-administrators) that can access the OpenShift AI dashboard. Starting with OpenShift AI 2.17, this functionality has moved to the **Auth** resource. If you have workflows (such as GitOps workflows) that interact with **OdhDashboardConfig**, you must update them to reference the **Auth** resource instead.

Table 6.1. Updated configurations

Resource	2.16 and earlier	2.17 and later versions
apiVersion	**opendatahub.io/v1alpha**	**services.platform.opendatahub.io/v1alpha1**
kind	**OdhDashboardConfig**	**Auth**
name	**odh-dashboard-config**	**auth**
Admin groups	**spec.groupsConfig.adminGroups**	**spec.adminGroups**
User groups	**spec.groupsConfig.allowedGroups**	**spec.allowedGroups**

6.1.9. Deprecated cluster configuration parameters

When using the CodeFlare SDK to run distributed workloads in Red Hat OpenShift AI, the following parameters in the Ray cluster configuration are now deprecated and should be replaced with the new parameters as indicated.

Deprecated parameter	Replaced by
head_cpus	**head_cpu_requests**, **head_cpu_limits**
head_memory	**head_memory_requests**, **head_memory_limits**
min_cpus	**worker_cpu_requests**

Deprecated parameter	Replaced by
max_cpus	**worker_cpu_limits**
min_memory	**worker_memory_requests**
max_memory	**worker_memory_limits**
head_gpus	**head_extended_resource_requests**
num_gpus	**worker_extended_resource_requests**

You can also use the new **extended_resource_mapping** and **overwrite_default_resource_mapping** parameters, as appropriate. For more information about these new parameters, see the CodeFlare SDK documentation (external).

6.2. REMOVED FUNCTIONALITY

Caikit-NLP component removed

The **caikit-nlp** component has been formally deprecated and removed from OpenShift AI 3.0.

This runtime is no longer included or supported in OpenShift AI. Users should migrate any dependent workloads to supported model serving runtimes.

TGIS component removed

The TGIS component, which was deprecated in OpenShift AI 2.19, has been removed in OpenShift AI 3.0.

TGIS continued to be supported through the OpenShift AI 2.16 Extended Update Support (EUS) lifecycle, which ended in June 2025.

Starting with this release, TGIS is no longer available or supported. Users should migrate their model serving workloads to supported runtimes such as Caikit or Caikit-TGIS.

AppWrapper Controller removed

The AppWrapper controller has been removed from OpenShift AI as part of the broader CodeFlare Operator removal process.

This change eliminates redundant functionality and reduces maintenance overhead and architectural complexity.

6.2.1. CodeFlare Operator removed

Starting with OpenShift AI 3.0, the CodeFlare Operator has been removed.

+ The functionality previously provided by the CodeFlare Operator is now included in the KubeRay Operator, which provides equivalent capabilities such as mTLS, network isolation, and authentication.

LAB-tuning feature removed

Starting with OpenShift AI 3.0, the LAB-tuning feature has been removed.

Users who previously relied on LAB-tuning for large language model customization should migrate to alternative fine-tuning or model customization methods.

Embedded Kueue component removed

The embedded Kueue component, which was deprecated in OpenShift AI 2.24, has been removed in OpenShift AI 3.0.
OpenShift AI now uses the Red Hat Build of the Kueue Operator to provide enhanced workload scheduling across distributed training, workbench, and model serving workloads.

The embedded Kueue component is not supported in any Extended Update Support (EUS) release.

Removal of DataSciencePipelinesApplication v1alpha1 API version

The **v1alpha1** API version of the **DataSciencePipelinesApplication** custom resource (**datasciencepipelinesapplications.opendatahub.io/v1alpha1**) has been removed.
OpenShift AI now uses the stable **v1** API version (**datasciencepipelinesapplications.opendatahub.io/v1**).

You must update any existing manifests or automation to reference the **v1** API version to ensure compatibility with OpenShift AI 3.0 and later.

6.2.2. Microsoft SQL Server command-line tool removal

Starting with OpenShift AI 2.24, the Microsoft SQL Server command-line tools (sqlcmd, bcp) have been removed from workbenches. You can no longer manage Microsoft SQL Server using the preinstalled command-line client.

6.2.3. Model registry ML Metadata (MLMD) server removal

Starting with OpenShift AI 2.23, the ML Metadata (MLMD) server has been removed from the model registry component. The model registry now interacts directly with the underlying database by using the existing model registry API and database schema. This change simplifies the overall architecture and ensures the long-term maintainability and efficiency of the model registry by transitioning from the **ml-metadata** component to direct database access within the model registry itself.

If you see the following error for your model registry deployment, this means that your database schema migration has failed:

```
error: error connecting to datastore: Dirty database version {version}. Fix and force version.
```

You can fix this issue by manually changing the database from a dirty state to 0 before traffic can be routed to the pod. Perform the following steps:

1. Find the name of your model registry database pod as follows:
   kubectl get pods -n <your-namespace> | grep model-registry-db

   Replace **<your-namespace>** with the namespace where your model registry is deployed.

2. Use **kubectl exec** to run the query on the model registry database pod as follows:
   kubectl exec -n <your-namespace> <your-db-pod-name> -c mysql -- mysql -u root -p"$MYSQL_ROOT_PASSWORD" -e "USE <your-db-name>; UPDATE schema_migrations SET dirty = 0;"

Replace **<your-namespace>** with your model registry namespace and **<your-db-pod-name>** with the pod name that you found in the previous step. Replace **<your-db-name>** with your model registry database name.

This will reset the dirty state in the database, allowing the model registry to start correctly.

6.2.4. Embedded subscription channel not used in some versions

For OpenShift AI 2.8 to 2.20 and 2.22 to 3.0, the **embedded** subscription channel is not used. You cannot select the **embedded** channel for a new installation of the Operator for those versions. For more information about subscription channels, see Installing the Red Hat OpenShift AI Operator .

6.2.5. Anaconda removal

Anaconda is an open source distribution of the Python and R programming languages. Starting with OpenShift AI version 2.18, Anaconda is no longer included in OpenShift AI, and Anaconda resources are no longer supported or managed by OpenShift AI.

If you previously installed Anaconda from OpenShift AI, a cluster administrator must complete the following steps from the OpenShift command-line interface to remove the Anaconda-related artifacts:

1. Remove the secret that contains your Anaconda password:
   oc delete secret -n redhat-ods-applications anaconda-ce-access

2. Remove the **ConfigMap** for the Anaconda validation cronjob:
   oc delete configmap -n redhat-ods-applications anaconda-ce-validation-result

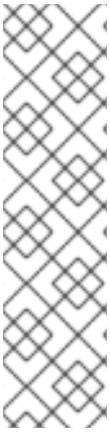
3. Remove the Anaconda image stream:
   oc delete imagestream -n redhat-ods-applications s2i-minimal-notebook-anaconda

4. Remove the Anaconda job that validated the downloading of images:
   oc delete job -n redhat-ods-applications anaconda-ce-periodic-validator-job-custom-run

5. Remove any pods related to Anaconda cronjob runs:
   oc get pods n redhat-ods-applications --no-headers=true | awk '/anaconda-ce-periodic-validator-job-custom-run*/'

6.2.6. Pipeline logs for Python scripts running in Elyra pipelines are no longer stored in S3

Logs are no longer stored in S3-compatible storage for Python scripts which are running in Elyra pipelines. From OpenShift AI version 2.11, you can view these logs in the pipeline log viewer in the OpenShift AI dashboard.



NOTE

For this change to take effect, you must use the Elyra runtime images provided in workbench images at version 2024.1 or later.

If you have an older workbench image version, update the **Version selection** field to a compatible workbench image version, for example, 2024.1, as described in Updating a project workbench.

Updating your workbench image version will clear any existing runtime image selections for your pipeline. After you have updated your workbench version, open your workbench IDE and update the properties of your pipeline to select a runtime image.

6.2.7. Beta subscription channel no longer used

Starting with OpenShift AI 2.5, the **beta** subscription channel is no longer used. You can no longer select the **beta** channel for a new installation of the Operator. For more information about subscription channels, see Installing the Red Hat OpenShift AI Operator .

6.2.8. HabanaAI workbench image removal

Support for the HabanaAI 1.10 workbench image has been removed. New installations of OpenShift AI from version 2.14 do not include the HabanaAI workbench image. However, if you upgrade OpenShift AI from a previous version, the HabanaAI workbench image remains available, and existing HabanaAI workbench images continue to function.

CHAPTER 7. RESOLVED ISSUES

The following notable issues are resolved in Red Hat OpenShift AI 3.0. Security updates, bug fixes, and enhancements for Red Hat OpenShift AI 3.0 are released as asynchronous errata. All OpenShift AI errata advisories are published on the Red Hat Customer Portal.

7.1. ISSUES RESOLVED IN RED HAT OPENSHIFT AI 3.0

RHOAIENG-37686 - Metrics not displayed on the Dashboard due to image name mismatch in runtime detection logic

Previously, metrics were not displayed on the OpenShift AI dashboard because digest-based image names were not correctly recognized by the runtime detection system. This issue affected all **InferenceService** deployments in OpenShift AI 2.25 and later. This issue has been resolved.

RHOAIENG-37492 - Dashboard console link not accessible on IBM Power in 3.0.0

Previously, on private cloud deployments running on IBM Power, the OpenShift AI dashboard link was not visible in the OpenShift console when the dashboard was enabled in the **DataScienceCluster** configuration. As a result, users could not access the dashboard through the console without manually creating a route. This issue has been resolved.

RHOAIENG-1152 - Basic workbench creation process fails for users who have never logged in to the dashboard

This issue is now obsolete as of OpenShift AI 3.0. The basic workbench creation process has been updated, and this behavior no longer occurs.

CHAPTER 8. KNOWN ISSUES

This section describes known issues in Red Hat OpenShift AI 3.0 and any known methods of working around these issues.

RHOAIENG-37228 - Manual DNS configuration required on OpenStack and private cloud environments

When deploying OpenShift AI 3.0 on OpenStack, CodeReady Containers (CRC), or other private cloud environments without integrated external DNS, external access to components such as the dashboard and workbenches might fail after installation. This occurs because the dynamically provisioned LoadBalancer Service does not automatically register its IP address in external DNS.

Workaround

To restore access, manually create the required A or CNAME records in your external DNS system. For instructions, see the Configuring External DNS for RHOAI 3.x on OpenStack and Private Clouds Knowledgebase article.

RHOAIENG-38658 - TrustyAI service issues during model inference with token authentication on IBM Z (s390x)

On IBM Z (s390x) architecture, the TrustyAI service encounters errors during model inference when token authentication is enabled. A **JsonParseException** displays while logging to the TrustyAI service logger, causing the bias monitoring process to fail or behave unexpectedly.

Workaround

Run the TrustyAI service without authentication. The issue occurs only when token authentication is enabled.

RHOAIENG-38333 - Code generated by the Generative AI Playground is invalid and required packages are missing from workbenches

Code automatically generated by the Generative AI Playground might cause syntax errors when run in OpenShift AI workbenches. Additionally, the **LlamaStackClient** package is not currently included in standard workbench images.

RHOAIENG-38263 - Intermittent failures with Guardrails Detector model on Hugging Face runtime for IBM Z

On IBM Z platforms, the Guardrails Detector model running on the Hugging Face runtime might intermittently fail to process identical requests. In some cases, a request that previously returned valid results fails with a parse error similar to the following example:

```
Invalid numeric literal at line 1, column 20
```

This error can cause the serving pod to temporarily enter a **CrashLoopBackOff** state, although it typically recovers automatically.

Workaround

None. The pod restarts automatically and resumes normal operation.

RHOAIENG-38253 - Distributed Inference Server with llm-d not listed on the Serving Runtimes page

While **Distributed Inference Server with llm-d**appears as an available option when deploying a model, it is not listed on the **Serving Runtimes** page under the **Settings** section.

This occurs because **Distributed Inference Server with llm-d**is a composite deployment type that includes additional components beyond a standard serving runtime. It therefore does not appear in the list of serving runtimes visible to administrators and cannot currently be hidden from end users.

Workaround

None. The **Distributed Inference Server with llm-d**option can still be used for model deployments, but it cannot be managed or viewed from the **Serving Runtimes** page.

RHOAIENG-38252 - **Model Registry Operator does not work with BYOIDC mode on OpenShift 4.20**

On OpenShift 4.20 clusters configured with Bring Your Own Identity Provider (BYOIDC) mode, deploying the Model Registry Operator fails.

When you create a **ModelRegistry** custom resource, it does not reach the **available: True** state. Instead, the resource shows a status similar to the following example:

```
status:
  conditions:
  - lastTransitionTime: "2025-11-06T22:09:04Z"
    message: 'unexpected reconcile error: failed to get API group resources: unable to retrieve the
complete list of server APIs: user.openshift.io/v1: the server could not find the requested resource'
    reason: DeploymentUnavailable
    status: "False"
    type: Available
```

Workaround

None.

You cannot create or deploy a Model Registry instance when using BYOIDC mode on OpenShift 4.20.

RHOAIENG-38180 - **Workbench requests to Feature Store service result in certificate errors**

When using the default configuration, the Feature Store (Feast) deployment is missing required certificates and a service endpoint. As a result, workbenches cannot send requests to the Feature Store by using the Feast SDK.

Workaround

Delete the existing **FeatureStore** custom resource (CR), then create a new one with the following configuration:

```
registry:
  local:
    server:
      restAPI: false
```

After the Feature Store pod starts running, edit the same CR to set **registry.local.server.restAPI: true** and save it without deleting the CR. Verify that both REST and gRPC services are created in your namespace, and wait for the pod to restart and become ready.

RHOAIENG-37916 - **LLM-D deployed model shows failed status on the Deployments page**

Models deployed using the {llm-d} initially display a **Failed** status on the **Deployments** page in the OpenShift AI dashboard, even though the associated pod logs report no errors or failures.

To confirm the status of the deployment, use the OpenShift console to monitor the pods in the project. When the model is ready, the OpenShift AI dashboard updates the status to **Started**.

Workaround

Wait for the model status to update automatically, or check the pod statuses in the OpenShift console to verify that the model has started successfully.

RHOAIENG-37882 – Custom workbench (AnythingLLM) fails to load

Deploying a custom workbench such as AnythingLLM 1.8.5 might fail to finish loading. Starting with OpenShift AI 3.0, all workbenches must be compatible with the Kubernetes Gateway API's path-based routing. Custom workbench images that do not support this requirement fail to load correctly.

Workaround

Update your custom workbench image to support path-based routing by serving all content from the **${NB_PREFIX}** path (for example, **/notebook/<namespace>/<workbench-name>**). Requests to paths outside this prefix (such as **/index.html** or **/api/data**) are not routed to the workbench container.

To fix existing workbenches:

- Update your application to handle requests at **${NB_PREFIX}/...** paths.
- Configure the base path in your framework, for example: **FastAPI(root_path=os.getenv('NB_PREFIX', ''))**
- Update nginx to preserve the prefix in redirects.
- Implement health endpoints returning HTTP 200 at: **${NB_PREFIX}/api**, **${NB_PREFIX}/api/kernels**, and **${NB_PREFIX}/api/terminals**.
- Use relative URLs and remove any hardcoded absolute paths such as **/menu**.

For more information, see the migration guide: Gateway API migration guide.

RHOAIENG-37855 – Model deployment from Model Catalog fails due to name length limit

When deploying certain models from the **Model Catalog**, the deployment might fail silently and remain in the **Starting** state. This issue occurs because KServe cannot create a deployment from the **InferenceService** when the resulting object name exceeds the 63-character limit.

Example

Attempting to deploy the model **RedHatAI/Mistral-Small-3.1-24B-Instruct-2503-FP8-dynamic** results in KServe trying to create a deployment named **isvc.redhataimistral-small-31-24b-instruct-2503-fp8-dynamic-predictor**, which has 69 characters and exceeds the maximum allowed length.

Workaround

Use shorter model names or rename the **InferenceService** to ensure the generated object name stays within the 63-character limit.

RHOAIENG-37842 – Ray workloads requiring ray.init() cannot be triggered outsid OpenShift AI

Ray workloads that require **ray.init()** cannot be triggered outside the OpenShift AI environment. These workloads must be submitted from within a workbench or pipeline running on OpenShift AI in OpenShift. Running these workloads externally is not supported and results in initialization failures.

Workaround

Run Ray workloads that call **ray.init()** only within an OpenShift AI workbench or pipeline context.

RHOAIENG-37743 - No progress bar displayed when starting workbenches

When starting a workbench, the **Progress** tab in the **Workbench Status** screen does not display step-by-step progress. Instead, it shows a generic message stating that "Steps may repeat or occur in a different order."

Workaround

To view detailed progress information, open the **Event Log** tab or use the OpenShift console to view the pod details associated with the workbench.

RHOAIENG-37667 - Model-as-a-Service (MaaS) available only for LLM-D runtime

Model-as-a-Service (MaaS) is currently supported only for models deployed with the **Distributed Inference Server with llm-d** runtime. Models deployed with the vLLM runtime cannot be served by MaaS at this time.

Workaround

None. Use the **llm-d** runtime for deployments that require Model-as-a-Service functionality.

RHOAIENG-37561 - Dashboard console link fails to access OpenShift AI on IBM Z clusters in 3.0.0

When attempting to access the OpenShift AI 3.0.0 dashboard using the console link on an IBM Z cluster, the connection fails.

Workaround

Create a route to the Gateway link by applying the following YAML file:

```
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: data-science-gateway-data-science-gateway-class
  namespace: openshift-ingress
spec:
  host: data-science-gateway.apps.<baseurl>
  port:
    targetPort: https
  tls:
    termination: passthrough
  to:
    kind: Service
    name: data-science-gateway-data-science-gateway-class
    weight: 100
  wildcardPolicy: None
```

RHOAIENG-37259 - Elyra Pipelines not supported on IBM Z (s390x)

Elyra Pipelines depend on Data Science Pipelines (DSP) for orchestration and validation. Because DSP is not currently available on IBM Z, Elyra pipeline-related functionality and tests are skipped.

Workaround

None. Elyra Pipelines will function correctly once DSP support is enabled and validated on IBM Z.

RHOAIENG-37015 - **TensorBoard reporting fails in PyTorch 2.8 training image**

When using TensorBoard reporting for training jobs that use the **SFTTrainer** with the image **registry.redhat.io/rhoai/odh-training-cuda128-torch28-py312-rhel9:rhoai-3.0**, or when the **report_to** parameter is omitted from the training configuration, the training job fails with a JSON serialization error.

Workaround

Install the latest versions of the **transformers** and **trl** packages and update the **torch_dtype** parameter to **dtype** in the training configuration.

If you are using the Training Operator SDK, you can specify the packages to install by using the **packages_to_install** parameter in the **create_job** function:

```
packages_to_install=[
    "transformers==4.57.1",
    "trl==0.24.0"
]
```

RHOAIENG-36757 - **Existing cluster storage option missing during model deployment when no connections exist**

When creating a model deployment in a project that has no data connections defined, the **Existing cluster storage** option is not displayed, even if suitable Persistent Volume Claims (PVCs) exist in the project. This prevents you from selecting an existing PVC for model deployment.

Workaround

Create at least one connection of type **URI** in the project to make the **Existing cluster storage** option appear.

RHOAIENG-31071 - **Parquet datasets not supported on IBM Z (s390x)**

Some built-in evaluation tasks, such as **arc_easy** and **arc_challenge**, use datasets provided by Hugging Face in Parquet format. Parquet is not supported on IBM Z.

Workaround

None. To evaluate models on IBM Z, use datasets in a supported format instead of Parquet.

RHAIENG-1795 - **CodeFlare with Ray does not work with Gateway**

When running the following commands, the output indicates that the Ray cluster has been created and is running, but the cell never completes because the Gateway route does not respond correctly:

```
cluster.up()
cluster.wait_ready()
```

As a result, subsequent operations such as fetching the Ray cluster or obtaining the job client fail, preventing job submission to the cluster.

Workaround

None. The Ray Dashboard Gateway route does not function correctly when created through CodeFlare.

RHAIENG-1796 - Pipeline name must be DNS compliant when using Kubernetes pipeline storage

When using Kubernetes as the storage backend for pipelines, Elyra does not automatically convert pipeline names to DNS-compliant values. If a non-DNS-compliant name is used when starting an Elyra pipeline, an error similar to the following appears:

```
[TIP: did you mean to set 'https://ds-pipeline-dspa-robert-tests.apps.test.rhoai.rh-aiservices-bu.com/pipeline' as the endpoint, take care not to include 's' at end]
```

Workaround

Use DNS-compliant names when creating or running Elyra pipelines.

RHAIENG-1139 - Cannot deploy LlamaStackDistribution with the same name in multiple namespaces

If you create two **LlamaStackDistribution** resources with the same name in different namespaces, the ReplicaSet for the second resource fails to start the Llama Stack pod. The Llama Stack Operator does not correctly assign security constraints when duplicate names are used across namespaces.

Workaround

Use a unique name for each **LlamaStackDistribution** in every namespace. For example, include the project name or add a suffix such as **llama-stack-distribution-209342**.

RHAIENG-1624 - Embeddings API timeout on disconnected clusters

On disconnected clusters, calls to the embeddings API might time out when using the default embedding model (**ibm-granite/granite-embedding-125m-english**) included in the default Llama Stack distribution image.

Workaround

Add the following environment variables to the **LlamaStackDistribution** custom resource to use the embedded model offline:

```
- name: SENTENCE_TRANSFORMERS_HOME
  value: /opt/app-root/src/.cache/huggingface/hub
- name: HF_HUB_OFFLINE
  value: "1"
- name: TRANSFORMERS_OFFLINE
  value: "1"
- name: HF_DATASETS_OFFLINE
  value: "1"
```

RHOAIENG-34923 - Runtime configuration missing when running a pipeline from JupyterLab

The runtime configuration might not appear in the Elyra pipeline editor when you run a pipeline from the first active workbench in a project. This occurs because the configuration fails to populate for the initial workbench session.

Workaround

Restart the workbench. After restarting, the runtime configuration becomes available for pipeline execution.

RHAIENG-35055 - Model catalog fails to initialize after upgrading from OpenShift AI 2.24

After upgrading from OpenShift AI 2.24, the model catalog might fail to initialize and load. The OpenShift AI dashboard displays a **Request access to model catalog**error.

Workaround

Delete the existing model catalog ConfigMap and deployment by running the following commands:

```
$ oc delete configmap model-catalog-sources -n rhoai-model-registries --ignore-not-found
$ oc delete deployment model-catalog -n rhoai-model-registries --ignore-not-found
```

RHAIENG-35529 - Reconciliation issues in Data Science Pipelines Operator when using external Argo Workflows

If you enable the embedded Argo Workflows controllers (**argoWorkflowsControllers: Managed**) before deleting an existing external Argo Workflows installation, the workflow controller might fail to start and the Data Science Pipelines Operator (DSPO) might not reconcile its custom resources correctly.

Workaround

Before enabling the embedded Argo Workflows controllers, delete any existing external Argo Workflows instance from the cluster.

RHAIENG-36756 - Existing cluster storage option missing during model deployment when no connections exist

When creating a model deployment in a project with no defined data connections, the **Existing cluster storage** option does not appear, even if Persistent Volume Claims (PVCs) are available. As a result, you cannot select an existing PVC for model storage.

Workaround

Create at least one connection of type **URI** in the project. Afterward, the **Existing cluster storage** option becomes available.

RHOAIENG-36817 - Inference server fails when Model server size is set tosmall

When creating an inference service via the dashboard, selecting a **small** Model server size causes subsequent inferencing requests to fail. As a result, the deployment of the inference service itself succeeds, but the inferencing requests fail with a timeout error.

Workaround

To resolve this issue, select the Model server size as **large** from the dropdown.

RHOAIENG-33995 - Deployment of an inference service for Phi and Mistral models fails

The creation of an inference service for Phi and Mistral models using vLLM runtime on IBM Power cluster with openshift-container-platform 4.19 fails due to an error related to CPU backend. As a result, deployment of these models is affected, causing inference service creation failure.

Workaround

To resolve this issue, disable the **sliding_window** mechanism in the serving runtime if it is enabled for CPU and Phi models. Sliding window is not currently supported in V1.

RHOAIENG-33795 – Manual Route creation needed for gRPC endpoint verification for Triton Inference Server on IBM Z

When verifying Triton Inference Server with gRPC endpoint, **Route** does not get created automatically. This happens because the Operator currently defaults to creating an edge-terminated route for REST only.

Workaround

To resolve this issue, manual Route creation is needed for gRPC endpoint verification for Triton Inference Server on IBM Z.

1. When the model deployment pod is up and running, define an edge-terminated **Route** object in a YAML file with the following contents:

```
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: <grpc-route-name>                 # e.g. triton-grpc
  namespace: <model-deployment-namespace>  # namespace where your model is
deployed
  labels:
    inferenceservice-name: <inference-service-name>
  annotations:
    haproxy.router.openshift.io/timeout: 30s
spec:
  host: <custom-hostname>                 # e.g. triton-grpc.<apps-domain>
  to:
    kind: Service
    name: <service-name>                  # name of the predictor service (e.g. triton-predictor)
    weight: 100
  port:
    targetPort: grpc                    # must match the gRPC port exposed by the service
  tls:
    termination: edge
  wildcardPolicy: None
```

2. Create the **Route** object:

```
oc apply -f <route-file-name>.yaml
```

3. To send an inference request, enter the following command:

```
grpcurl -cacert <ca_cert_file>\ 1
  -protoset triton_desc.pb \
  -d '{
    "model_name": "<model_name>",
    "inputs": [
      {
        "name": "<input_tensor_name>",
        "shape": [<shape>],
        "datatype": "<data_type>",
        "contents": {
```

```
          "<datatype_specific_contents>": [<input_data_values>]
        }
      }
    ],
    "outputs": [
      {
        "name": "<output_tensor_name>"
      }
    ]
}' \
<grpc_route_host>:443 \
inference.GRPCInferenceService/ModelInfer
```

1 <ca_cert_file> is the path to your cluster router CA cert (for example, router-ca.crt).

NOTE

<triton_protoset_file> is compiled as a protobuf descriptor file. You can generate it as **protoc -I. --descriptor_set_out=triton_desc.pb --include_imports grpc_service.proto**.

Download **grpc_service.proto** and **model_config.proto** files from the triton-inference-service GitHub page.

RHOAIENG-33697 - Unable to Edit or Delete models unless status is "Started"

When you deploy a model on the NVIDIA NIM or single-model serving platform, the **Edit** and **Delete** options in the action menu are not available for models in the **Starting** or **Pending** states. These options become available only after the model has been successfully deployed.

Workaround

Wait until the model is in the **Started** state to make any changes or to delete the model.

RHOAIENG-33645 - LM-Eval Tier1 test failures

There can be failures with LM-Eval Tier1 tests because **confirm_run_unsafe_code** is not passed as an argument when a job is run, if you are using an older version of the **trustyai-service-operator**.

Workaround

Ensure that you are using the latest version of the **trustyai-service-operator** and that **AllowCodeExecution** is enabled.

RHOAIENG-29729 - Model registry Operator in a restart loop after upgrade

After upgrading from OpenShift AI version 2.22 or earlier to version 2.23 or later with the model registry component enabled, the model registry Operator might enter a restart loop. This is due to an insufficient memory limit for the manager container in the **model-registry-operator-controller-manager** pod.

Workaround

To resolve this issue, you must trigger a reconciliation for the **model-registry-operator-controller-manager** deployment. Adding the **opendatahub.io/managed='true'** annotation to the deployment will accomplish this and apply the correct memory limit. You can add the annotation by running the

following command:

```
oc annotate deployment model-registry-operator-controller-manager -n redhat-ods-applications
opendatahub.io/managed='true' --overwrite
```

NOTE

This command overwrites custom values in the **model-registry-operator-controller-manager** deployment. For more information about custom deployment values, see Customizing component deployment resources.

After the deployment updates and the memory limit increases from 128Mi to 256Mi, the container memory usage will stabilize and the restart loop will stop.

RHOAIENG-31238 - New observability stack enabled when creating DSCInitialization

When you remove a DSCInitialization resource and create a new one using OpenShift AI console form view, it enables a Technology Preview observability stack. This results in the deployment of an unwanted observability stack when recreating a DSCInitialization resource.

Workaround

To resolve this issue, manually remove the "metrics" and "traces" fields when recreating the DSCInitiliazation resource using the form view.
This is not required if you want to use the Technology Preview observability stack.

RHOAIENG-32599 - Inference service creation fails on IBM Z cluster

When you attempt to create an inference service using the vLLM runtime on an IBM Z cluster, it fails with the following error: **ValueError: 'aimv2' is already used by a Transformers config, pick another name**.

Workaround

None.

RHOAIENG-29731 - Inference service creation fails on IBM Power cluster with OpenShift 4.19

When you attempt to create an inference service by using the vLLM runtime on an IBM Power cluster on OpenShift Container Platform version 4.19, it fails due to an error related to Non-Uniform Memory Access (NUMA).

Workaround

When you create an inference service, set the environment variable **VLLM_CPU_OMP_THREADS_BIND** to **all**.

RHOAIENG-29292 - vLLM logs permission errors on IBM Z due to usage stats directory access

When running vLLM on the IBM Z architecture, the inference service starts successfully, but logs an error in a background thread related to usage statistics reporting. This happens because the service tries to write usage data to a restricted location (/**.config**), which it does not have permission to access.

The following error appears in the logs:

```
Exception in thread Thread-2 (_report_usage_worker):
Traceback (most recent call last):
 ...
PermissionError: [Error 13] Permission denied: '/.config'
```

Workaround

To prevent this error and suppress the usage stats logging, set the **VLLM_NO_USAGE_STATS=1** environment variable in the inference service deployment. This disables automatic usage reporting, avoiding permission issues when you write to system directories.

RHOAIENG-24545 - Runtime images are not present in the workbench after the first start

The list of runtime images does not properly populate the first running workbench instance in the namespace, therefore no image is shown for selection in the Elyra pipeline editor.

Workaround

Restart the workbench. After restarting the workbench, the list of runtime images populates both the workbench and the select box for the Elyra pipeline editor.

RHOAIENG-20209 - Warning message not displayed when requested resources exceed threshold

When you click **Distributed workloads → Project metrics** and view the **Requested resources** section, the charts show the requested resource values and the total shared quota value for each resource (**CPU** and **Memory**). However, when the **Requested by all projects** value exceeds the **Warning threshold** value for that resource, the expected warning message is not displayed.

Workaround

None.

SRVKS-1301 (previously documented as RHOAIENG-18590) - The KnativeServing resource fails after disabling and enabling KServe

After disabling and enabling the **kserve** component in the DataScienceCluster, the **KnativeServing** resource might fail.

Workaround

Delete all **ValidatingWebhookConfiguration** and **MutatingWebhookConfiguration** webhooks related to Knative:

1. Get the webhooks:

   ```
   oc get ValidatingWebhookConfiguration,MutatingWebhookConfiguration | grep -i knative
   ```

2. Ensure KServe is disabled.
3. Get the webhooks:

   ```
   oc get ValidatingWebhookConfiguration,MutatingWebhookConfiguration | grep -i knative
   ```

4. Delete the webhooks.
5. Enable KServe.

6. Verify that the KServe pod can successfully spawn, and that pods in the **knative-serving** namespace are active and operational.

RHOAIENG-16247 - Elyra pipeline run outputs are overwritten when runs are launched from OpenShift AI dashboard

When a pipeline is created and run from Elyra, outputs generated by the pipeline run are stored in the folder **bucket-name/pipeline-name-timestamp** of object storage.

When a pipeline is created from Elyra and the pipeline run is started from the OpenShift AI dashboard, the timestamp value is not updated. This can cause pipeline runs to overwrite files created by previous pipeline runs of the same pipeline.

This issue does not affect pipelines compiled and imported using the OpenShift AI dashboard because **runid** is always added to the folder used in object storage. For more information about storage locations used in AI pipelines, see Storing data with pipelines.

Workaround

When storing files in an Elyra pipeline, use different subfolder names on each pipeline run.

OCPBUGS-49422 - AMD GPUs and AMD ROCm workbench images are not supported in a disconnected environment

This release of OpenShift AI does not support AMD GPUs and AMD ROCm workbench images in a disconnected environment because installing the AMD GPU Operator requires internet access to fetch dependencies needed to compile GPU drivers.

Workaround

None.

RHOAIENG-7716 - Pipeline condition group status does not update

When you run a pipeline that has loops (**dsl.ParallelFor**) or condition groups (**dsl.If**), the UI displays a Running status for the loops and groups, even after the pipeline execution is complete.

Workaround

You can confirm if a pipeline is still running by checking that no child tasks remain active.

1. From the OpenShift AI dashboard, click **Develop & train → Pipelines → Runs**.
2. From the **Project** list, click your data science project.
3. From the **Runs** tab, click the pipeline run that you want to check the status of.
4. Expand the condition group and click a child task.
   A panel that contains information about the child task is displayed
5. On the panel, click the **Task** details tab.
   The **Status** field displays the correct status for the child task.

RHOAIENG-6409 - **Cannot save parameter** errors appear in pipeline logs for successful runs

When you run a pipeline more than once, **Cannot save parameter** errors appear in the pipeline logs for successful pipeline runs. You can safely ignore these errors.

Workaround

None.

RHOAIENG-3025 - OVMS expected directory layout conflicts with the KServe StoragePuller layout

When you use the OpenVINO Model Server (OVMS) runtime to deploy a model on the single-model serving platform (which uses KServe), there is a mismatch between the directory layout expected by OVMS and that of the model-pulling logic used by KServe. Specifically, OVMS requires the model files to be in the **/<mnt>/models/1/** directory, while KServe places them in the **/<mnt>/models/** directory.

Workaround

Perform the following actions:

1. In your S3-compatible storage bucket, place your model files in a directory called **1/**, for example, **/<s3_storage_bucket>/models/1/<model_files>**.

2. To use the OVMS runtime to deploy a model on the single-model serving platform, choose one of the following options to specify the path to your model files:

   - If you are using the OpenShift AI dashboard to deploy your model, in the **Path** field for your data connection, use the **/<s3_storage_bucket>/models/** format to specify the path to your model files. Do not specify the **1/** directory as part of the path.

   - If you are creating your own **InferenceService** custom resource to deploy your model, configure the value of the **storageURI** field as **/<s3_storage_bucket>/models/**. Do not specify the **1/** directory as part of the path.

KServe pulls model files from the subdirectory in the path that you specified. In this case, KServe correctly pulls model files from the **/<s3_storage_bucket>/models/1/** directory in your S3-compatible storage.

RHOAIENG-3018 - OVMS on KServe does not expose the correct endpoint in the dashboard

When you use the OpenVINO Model Server (OVMS) runtime to deploy a model on the single-model serving platform, the URL shown in the **Inference endpoint** field for the deployed model is not complete.

Workaround

To send queries to the model, you must add the **/v2/models/_<model-name>_/infer** string to the end of the URL. Replace **_<model-name>_** with the name of your deployed model.

RHOAIENG-2228 - The performance metrics graph changes constantly when the interval is set to 15 seconds

On the **Endpoint performance** tab of the model metrics screen, if you set the **Refresh interval** to 15 seconds and the **Time range** to 1 hour, the graph results change continuously.

Workaround

None.

RHOAIENG-2183 - Endpoint performance graphs might show incorrect labels

In the **Endpoint performance** tab of the model metrics screen, the graph tooltip might show incorrect labels.

Workaround

None.

RHOAIENG-131 - gRPC endpoint not responding properly after the InferenceService reports as Loaded

When numerous **InferenceService** instances are generated and directed requests, Service Mesh Control Plane (SMCP) becomes unresponsive. The status of the **InferenceService** instance is **Loaded**, but the call to the gRPC endpoint returns with errors.

Workaround

Edit the **ServiceMeshControlPlane** custom resource (CR) to increase the memory limit of the Istio egress and ingress pods.

RHOAIENG-1619 (previously documented as DATA-SCIENCE-PIPELINES-165) - Poor error message when S3 bucket is not writable

When you set up a data connection and the S3 bucket is not writable, and you try to upload a pipeline, the error message **Failed to store pipelines** is not helpful.

Workaround

Verify that your data connection credentials are correct and that you have write access to the bucket you specified.

RHOAIENG-1207 (previously documented asODH-DASHBOARD-1758) - Error duplicating OOTB custom serving runtimes several times

If you duplicate a model-serving runtime several times, the duplication fails with the **Serving runtime name "<name>" already exists** error message.

Workaround

Change the **metadata.name** field to a unique value.

RHOAIENG-133 - Existing workbench cannot run Elyra pipeline after workbench restart

If you use the Elyra JupyterLab extension to create and run pipelines within JupyterLab, and you configure the pipeline server *after* you created a workbench and specified a workbench image within the workbench, you cannot execute the pipeline, even after restarting the workbench.

Workaround

1. Stop the running workbench.
2. Edit the workbench to make a small modification. For example, add a new dummy environment variable, or delete an existing unnecessary environment variable. Save your changes.
3. Restart the workbench.
4. In the left sidebar of JupyterLab, click **Runtimes**.
5. Confirm that the default runtime is selected.

RHODS-12798 - Pods fail with "unable to init seccomp" error

Pods fail with **CreateContainerError** status or **Pending** status instead of **Running** status, because of a known kernel bug that introduced a **seccomp** memory leak. When you check the events on the namespace where the pod is failing, or run the **oc describe pod** command, the following error appears:

> runc create failed: unable to start container process: unable to init seccomp: error loading seccomp filter into kernel: error loading seccomp filter: errno 524

Workaround

Increase the value of **net.core.bpf_jit_limit** as described in the Red Hat Knowledgebase solution Pods failing with error loading seccomp filter into kernel: errno 524 in OpenShift 4 .

KUBEFLOW-177 - Bearer token from application not forwarded by OAuth-proxy

You cannot use an application as a custom workbench image if its internal authentication mechanism is based on a bearer token. The OAuth-proxy configuration removes the bearer token from the headers, and the application cannot work properly.

Workaround

None.

KUBEFLOW-157 - Logging out of JupyterLab does not work if you are already logged out of the OpenShift AI dashboard

If you log out of the OpenShift AI dashboard before you log out of JupyterLab, then logging out of JupyterLab is not successful. For example, if you know the URL for a Jupyter notebook, you are able to open this again in your browser.

Workaround

Log out of JupyterLab before you log out of the OpenShift AI dashboard.

RHODS-7718 - User without dashboard permissions is able to continue using their running workbenches indefinitely

When a Red Hat OpenShift AI administrator revokes a user's permissions, the user can continue to use their running workbenches indefinitely.

Workaround

When the OpenShift AI administrator revokes a user's permissions, the administrator should also stop any running workbenches for that user.

RHODS-5543 - When using the NVIDIA GPU Operator, more nodes than needed are created by the Node Autoscaler

When a pod cannot be scheduled due to insufficient available resources, the Node Autoscaler creates a new node. There is a delay until the newly created node receives the relevant GPU workload. Consequently, the pod cannot be scheduled and the Node Autoscaler's continuously creates additional new nodes until one of the nodes is ready to receive the GPU workload. For more information about this issue, see the Red Hat Knowledgebase solution When using the NVIDIA GPU Operator, more nodes than needed are created by the Node Autoscaler.

Workaround

Apply the **cluster-api/accelerator** label in **machineset.spec.template.spec.metadata**. This causes the autoscaler to consider those nodes as unready until the GPU driver has been deployed.

RHODS-4799 - Tensorboard requires manual steps to view

When a user has TensorFlow or PyTorch workbench images and wants to use TensorBoard to display data, manual steps are necessary to include environment variables in the workbench environment, and to import those variables for use in your code.

Workaround

When you start your basic workbench, use the following code to set the value for the TENSORBOARD_PROXY_URL environment variable to use your OpenShift AI user ID.

```
import os
os.environ["TENSORBOARD_PROXY_URL"]= os.environ["NB_PREFIX"]+"/proxy/6006/"
```

RHODS-4718 - The Intel® oneAPI AI Analytics Toolkits quick start references nonexistent sample notebooks

The Intel® oneAPI AI Analytics Toolkits quick start, located on the **Resources** page on the dashboard, requires the user to load sample notebooks as part of the instruction steps, but refers to notebooks that do not exist in the associated repository.

Workaround

None.

RHOAING-1147 (previously documented as RHODS-2881) - Actions on dashboard not clearly visible

The dashboard actions to revalidate a disabled application license and to remove a disabled application tile are not clearly visible to the user. These actions appear when the user clicks on the application tile's **Disabled** label. As a result, the intended workflows might not be clear to the user.

Workaround

None.

RHODS-2096 - IBM Watson Studio not available in OpenShift AI

IBM Watson Studio is not available when OpenShift AI is installed on OpenShift Dedicated 4.9 or higher, because it is not compatible with these versions of OpenShift Dedicated.

Workaround

Contact the Red Hat Customer Portal for assistance with manually configuring Watson Studio on OpenShift Dedicated 4.9 and higher.

CHAPTER 9. PRODUCT FEATURES

Red Hat OpenShift AI provides a rich set of features for data scientists and cluster administrators. To learn more, see Introduction to Red Hat OpenShift AI.